

Brezžična in Mobilna Omrežja
Študijsko leto 2025/2026

Ranljivosti Bluetooth naprav

Končno poročilo seminarske naloge

Žan Perat
Vpisna št. 63230239
Klemen Pregelj
Vpisna št. 63230263

Ljubljana, 9. maj 2026

Kazalo

1	Uvod in motivacija	3
2	Moč signala glede na oddaljenost in število naprav	4
2.1	Moč signala v prostoru	4
2.2	Vpliv števila naprav na moč signala	5
3	L2ping in analiza prometa	6
3.1	Analiza prometa v Wiresharku	8
3.1.1	Vzpostavitev povezave	8
3.1.2	Izmenjava zmogljivosti in identifikacija naprave	9
3.1.3	Vzpostavitev L2CAP komunikacije in prenos podatkov	10
3.1.4	Zaključek povezave	11
4	Tipi napadov na bluetooth naprave	13
4.1	BlueSnarfing	13
4.2	MAC address spoofing	13
4.3	BlueBorne	13
4.4	Reflection Attack	13
4.5	Denial of Service (DoS)	14
4.6	Bluejacking	14
4.7	Bluesmacking	14
4.8	PIN Brute Force	14
5	Eksperimentalna analiza Bluetooth napadov	15

5.1	Hcitol	15
5.2	Bluetoothctl	15
5.3	Sdptool	16
5.4	Še nekaj osnovnih ukazov	17
5.5	CVE-2023-45866	18
5.5.1	Rezultati testiranja	19
5.5.2	Ugotovitve	20
5.6	Denial of Service (DoS)	21
5.7	Bluetooth spam	22
6	Zaščita pred Bluetooth napadi	25
6.1	Splošni ukrepi	25
6.2	Specifične zaščite glede na vrsto napada	26
6.3	Posebnosti za starejše naprave (Bluetooth 3.0 in nižje)	26
7	Zaključek	28

1 Uvod in motivacija

Bluetooth tehnologija je danes vgrajena v ogromno število naprav – od pametnih telefonov in prenosnikov do brezžičnih slušalk, mikrofонов in pametnih ur. Kljub njeni razširjenosti pa se marsikdo ne zaveda varnostnih tveganj, ki jih prinaša. Do sedaj si še nisem vzel časa, da bi se poglobljeno ukvarjal z Bluetooth tehnologijo in njenimi ranljivostmi. Zato sem se odločil, da to področje temeljito raziščem in pri tem pridobim nova znanja ter praktične izkušnje.

V okviru te seminarske naloge sem se srečal z različnimi orodji za delo z Bluetooth napravami, kot so `hciconfig`, `hcitool`, `l2ping` in `bluesnarfer`. Naučil sem se iskati bližnje naprave (mikrofone, slušalke, pametne telefone) ter meriti moč in kakovost signala glede na oddaljenost in število povezanih naprav. Z ukazom `l2ping` sem generiral L2CAP echo promet in meril zakasnitve (latenco) za posamezne naprave ter preverjal, kako dodaten promet vpliva na njihovo delovanje. Generirani promet sem zajel in analiziral z orodjem Wireshark [2].

Poleg meritev sem preučil tudi različne tipe Bluetooth napadov. V praktičnem delu sem demonstriral tudi različne vrste napadov in preveril, kakšni napadi so sploh možni. Na koncu sem opisal še različne načine zaščite Bluetooth povezav. Vsi eksperimenti so potekali izključno na lastnih napravah, kar mi je omogočilo varno in nadzorovano testiranje.

Vsa izvorna koda in zajeti podatki so dostopni v Github repozitoriju: <https://github.com/zanzan731/SeminarskaBMO>

Cilj naloge ni bil le opraviti predpisane meritve, ampak predvsem razumeti ozadje delovanja Bluetootha in se naučiti prepoznati ter preprečiti morebitne napade.

2 Moč signala glede na oddaljenost in število naprav

V tem poglavju analiziramo odvisnost moči brezžičnega signala (RSSI) od oddaljenosti ter vpliv števila aktivnih naprav v prostoru.

Vse meritve so bile izvedene večkrat z namenom zmanjšanja vpliva naključnih odstopanj. Vsaka posamezna meritev je bila ponovljena petkrat, končna vrednost RSSI pa je bila izračunana kot povprečje teh meritev.

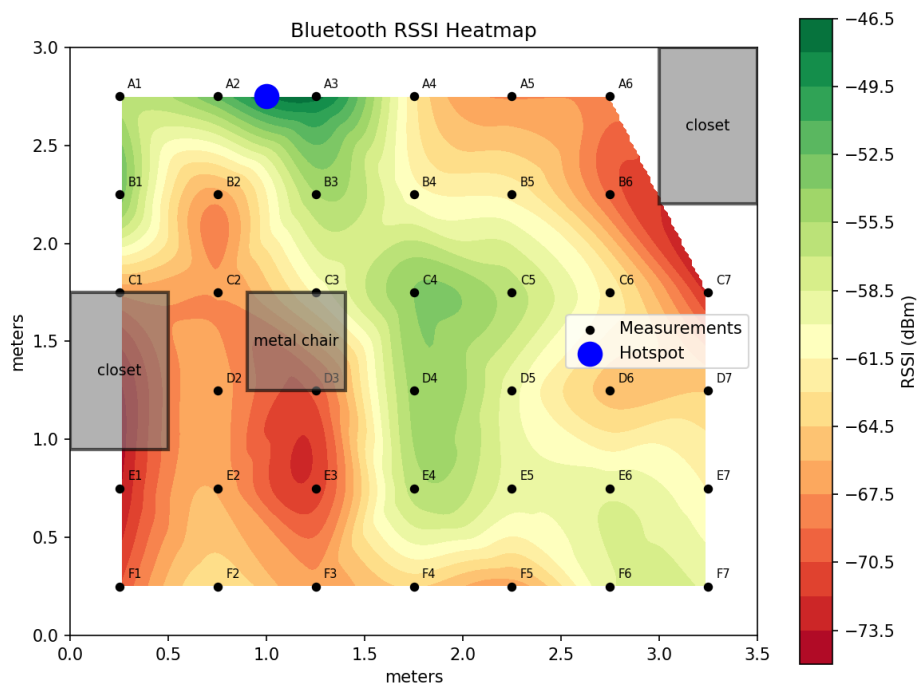
Za zajem podatkov smo uporabili Python skripto z uporabo knjižnice **Bleak**, ki omogoča skeniranje BLE naprav. Naslednja vrstica v Pythonu:

```
results = await BleakScanner.discover(timeout=5.0, return_adv=True)
```

vrne slovar najdenih naprav, iz katerega odčitamo RSSI iskane naprave.

2.1 Moč signala v prostoru

Na sliki 1 je prikazana toplotna karta (angl. heatmap) jakosti signala v prostoru dimenzij $3 \times 3,5$ m. Dostopna točka, označena z modro, se nahaja med točkama A2 in A3. Opazimo, da je signal najmočnejši v njeni bližini ter postopoma slabi z naraščajočo oddaljenostjo. Na porazdelitev moči pomembno vplivajo tudi ovire in odboji od sten, kar se kaže v območjih z nenadnimi padci signala. Predmeti, ki najbolj vplivajo na moč signala so označeni na sliki.



Slika 1: RSSI toplotna karta v prostoru

2.2 Vpliv števila naprav na moč signala

Ali se RSSI spremeni, če v okolico naše bluetooth povezave dodajamo naprave, na katere generiramo bluetooth promet?

```
\\Ukaz za generiranje prometa
sudo l2ping <MAC address>
```

Bluetooth uporablja t.i. **adaptive frequency hopping (AFH)**, kar pomeni, da naprave, ob pošiljanju paketov, izjemno hitro spreminjajo frekvenčne kanale (BLE uporablja 40 kanalov) in se izogibajo trkom in šumu. Zaradi tega se RSSI ne spremeni. Razlika ob povečanju prometa je ta, da se zaradi trkov izgubi več paketov, posledično pa pride do večje latence.

3 L2ping in analiza prometa

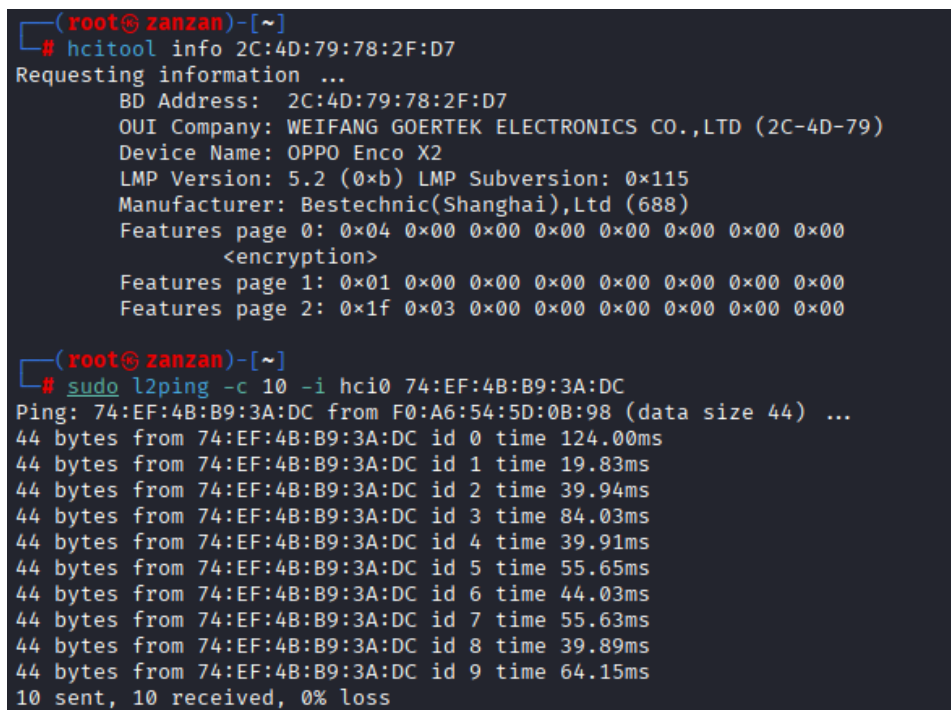
V tem delu se bomo spoznali z l2ping in preučili osnove bluetootha.

l2ping je orodje, ki je del paketa BlueZ (Bluetooth sklad za Linux). Omogoča pošiljanje **L2CAP echo zahtevkov** (podobno kot klasični ping v IP omrežjih) na izbrano Bluetooth napravo. Uporablja se za preverjanje dosegljivosti naprave, merjenje odzivnega časa (latence) in testiranje stabilnosti Bluetooth povezave na nivoju protokola L2CAP.

Najprej bomo testirali l2ping z različnimi napravami in sicer OPPO Find X5, Samsung Galaxy S7 edge, Samsung Galaxy Ace S5830, OPPO Enco X2. Za vsako napravo sem uporabil ukaze:

```
\\Za bluetooth verzijo  
hcitool info <MAC address>
```

```
\\Za generiranje ping paketov in merjenje latence  
sudo l2ping -c 10 -i hci0 <MAC address>
```



```
(root@zanzan)-[~]  
# hcitool info 2C:4D:79:78:2F:D7  
Requesting information ...  
BD Address: 2C:4D:79:78:2F:D7  
OUI Company: WEIFANG GOERTEK ELECTRONICS CO.,LTD (2C-4D-79)  
Device Name: OPPO Enco X2  
LMP Version: 5.2 (0xb) LMP Subversion: 0x115  
Manufacturer: Bestechnic(Shanghai),Ltd (688)  
Features page 0: 0x04 0x00 0x00 0x00 0x00 0x00 0x00 0x00  
               <encryption>  
Features page 1: 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00  
Features page 2: 0x1f 0x03 0x00 0x00 0x00 0x00 0x00 0x00  
  
(root@zanzan)-[~]  
# sudo l2ping -c 10 -i hci0 74:EF:4B:B9:3A:DC  
Ping: 74:EF:4B:B9:3A:DC from F0:A6:54:5D:0B:98 (data size 44) ...  
44 bytes from 74:EF:4B:B9:3A:DC id 0 time 124.00ms  
44 bytes from 74:EF:4B:B9:3A:DC id 1 time 19.83ms  
44 bytes from 74:EF:4B:B9:3A:DC id 2 time 39.94ms  
44 bytes from 74:EF:4B:B9:3A:DC id 3 time 84.03ms  
44 bytes from 74:EF:4B:B9:3A:DC id 4 time 39.91ms  
44 bytes from 74:EF:4B:B9:3A:DC id 5 time 55.65ms  
44 bytes from 74:EF:4B:B9:3A:DC id 6 time 44.03ms  
44 bytes from 74:EF:4B:B9:3A:DC id 7 time 55.63ms  
44 bytes from 74:EF:4B:B9:3A:DC id 8 time 39.89ms  
44 bytes from 74:EF:4B:B9:3A:DC id 9 time 64.15ms  
10 sent, 10 received, 0% loss
```

Slika 2: Primer hcitool in l2ping

Naprava	Bluetooth verzija	Min (ms)	Avg (ms)	Max (ms)
OPPO Find X5	5.3	19.83	56.71	124.00
Samsung Galaxy S7 edge	4.2	11.95	35.35	39.83
Samsung Galaxy Ace S5830	3.0	15.43	41.34	79.89
OPPO Enco X2	5.2	39.24	50.72	102.58

Tabela 1: Meritve latence z orodjem `l2ping`

Ta tabela prikazuje odgovore na ping pakete v idelnem okolju. Za generiranje velikega števila paketov oziroma za zelo preprosti flood napad sem upoabil ukaz:

```
sudo l2ping -c 100 -i hci0 -f <MAC address>
```

Ta ukaz je precej podoben prejšnjemu za razliko od večjega števila paketov in dodatno zastavico `-f`, ki pomeni flood (pošlji pakete brez zakasnitve). Tukaj je tudi računanje povprečja precej nadležno, saj `l2ping` ne sam izpisuje teh podatkov zato sem si pomagal z naslednjim python programom:

```

1  import subprocess
2  import re
3
4  def ping_bluetooth(mac_address, count=100, device="hci0"):
5      cmd = ['l2ping', '-c', str(count), '-i', str(device), '-f',
6             ↪ mac_address]
7      result = subprocess.run(cmd, capture_output=True, text=True)
8
9      times = []
10     for line in result.stdout.split('\n'):
11         match = re.search(r'time\s+([\d\.]+)ms', line)
12         if match:
13             times.append(float(match.group(1)))
14
15     if times:
16         print(f"Min: {min(times):.2f} ms")
17         print(f"Avg: {sum(times)/len(times):.2f} ms")
18         print(f"Max: {max(times):.2f} ms")
19     else:
20         print("No ping responses received.")
21
22 ping_bluetooth('70:28:8B:B8:29:57')
```


Naprava	Bluetooth verzija	Min (ms)	Avg (ms)	Max (ms)
OPPO Find X5	5.3	11.66	69.37	299.87
Samsung Galaxy S7 edge	4.2	11.98	42.02	71.87
Samsung Galaxy Ace S5830	3.0	11.64	40.11	80.09
OPPO Enco X2	5.2	31.71	50.51	103.80

Tabela 2: Meritve latence z orodjem `l2ping flood`

Kot vidimo se vrednosti niso dosti povečale. Mogoče boste rekli da je 100 paketov premalo, zato sem preizkusil tudi z 2000 in 10000 paketov. Rezultati so bili približno enaki, povprečje mogoče za 10 ms višje ampak nobena naprava ni kazala nobenih znakov DoS napada. Največja zakasnitev paketa je imela naprava OPPO Find X5 pri 10000 paketih z 420.03 ms ampak povprečje je celo pri tej komunikaciji ostalo na 84.16 ms. Pri nobeni komunikaciji ni bil izgubljen niti en paket.

3.1 Analiza prometa v Wiresharku

Analizo prometa sem izvajal na napravi OPPO Find X5 z uporabo 10 ping paketov. Da lahko razumemo, kako se povezava vzpostavi in kako se ti paketi prenašajo, moramo najprej razumeti osnovno delovanje Bluetooth komunikacije.

3.1.1 Vzpostavitev povezave

Vzpostavitev povezave se zgodi na začetku komunikacije in je prikazana na sliki 3.

1 0.000000	host	controller	HCI_CMD	17 Sent Create Connection
2 0.251001	controller	host	HCI_EVT	7 Rcvd Command Status (Create Connection)
3 0.700325	controller	host	HCI_EVT	11 Rcvd Role Change
4 0.709191	controller	host	HCI_EVT	14 Rcvd Connect Complete

Slika 3: Vzpostavitev povezave

Prvi paket, **Create Connection**, predstavlja zahtevo, ki jo moja naprava pošlje Bluetooth krmilniku z namenom vzpostavitve povezave s ciljno napravo (telefonom). Ta paket predstavlja začetno točko tako imenovanega “handshake” procesa.

Sledijo odzivi krmilnika, ki potrjujejo in vzpostavljajo povezavo. Paket **Command Status** potrdi, da je krmilnik uspešno sprejel zahtevo za vzpo-

stavitev povezave. Nato paket **Role Change** določi vloge naprav v povezavi, kjer ena naprava prevzame vlogo *master*, druga pa *slave*. Naprava v vlogi *master* upravlja komunikacijo, določa časovne intervale in usklajuje izmenjavo podatkov, medtem ko *slave* sledi njenim zahtevam ter odgovarja na prejete ukaze.

Na koncu paket **Connection Complete** potrdi, da je bila fizična povezava med napravama uspešno vzpostavljena in da sta napravi pripravljeni na nadaljnjo komunikacijo.

3.1.2 Izmenjava zmogljivosti in identifikacija naprave

Po uspešni vzpostavitvi fizične povezave sledi faza izmenjave zmogljivosti med napravama. V tej fazi si napravi izmenjata informacije o podprtih funkcionalnostih in lastnostih povezave.

5 0.709287	host	controller	HCI_CMD	6 Sent Read Remote Supported Features
6 0.709689	controller	host	HCI_EVT	7 Rcvd Command Status (Read Remote Supported Features)
7 0.709813	controller	host	HCI_EVT	14 Rcvd Read Remote Supported Features
8 0.709866	host	controller	HCI_CMD	7 Sent Read Remote Extended Features
9 0.718441	controller	host	HCI_EVT	7 Rcvd Command Status (Read Remote Extended Features)
10 0.718617	controller	host	HCI_EVT	16 Rcvd Read Remote Extended Features Complete
11 0.718852	host	controller	HCI_CMD	14 Sent Remote Name Request
12 0.718865	localhost ()	OppoMobileTe_b9:3a:...	L2CAP	15 Sent Information Request (Extended Features Mask)
13 0.711436	controller	host	HCI_EVT	7 Rcvd Command Status (Remote Name Request)
14 0.745683	controller	host	HCI_EVT	6 Rcvd Max Slots Change
15 0.821916	controller	host	HCI_EVT	8 Rcvd Connection Packet Type Changed
16 0.826622	controller	host	HCI_EVT	7 Rcvd Link Supervision Timeout Changed
17 0.881592	controller	host	HCI_EVT	258 Rcvd Remote Name Request Complete
18 0.881863	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
19 0.891939	host	controller	HCI_CMD	6 Sent Read RSSI
20 0.892217	controller	host	HCI_EVT	10 Rcvd Command Complete (Read RSSI)
21 0.892318	host	controller	HCI_CMD	7 Sent Read Tx Power Level
22 0.892586	controller	host	HCI_EVT	10 Rcvd Command Complete (Read Tx Power Level)
23 0.941492	controller	host	HCI_EVT	8 Rcvd Connection Packet Type Changed
24 0.958445	OppoMobileTe_b9:3a:...	localhost ()	L2CAP	15 Rcvd Information Request (Extended Features Mask)
25 0.958953	OppoMobileTe_b9:3a:...	localhost ()	L2CAP	21 Rcvd Information Response (Extended Features Mask, Success)
26 0.958975	localhost ()	OppoMobileTe_b9:3a:...	L2CAP	21 Sent Information Response (Extended Features Mask, Success)
27 0.958988	localhost ()	OppoMobileTe_b9:3a:...	L2CAP	15 Sent Information Request (Fixed Channels Supported)
28 0.966475	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
29 0.966965	OppoMobileTe_b9:3a:...	localhost ()	L2CAP	15 Rcvd Information Request (Fixed Channels Supported)
30 0.966948	localhost ()	OppoMobileTe_b9:3a:...	L2CAP	25 Sent Information Response (Fixed Channels Supported, Success)
31 0.966443	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
32 0.967729	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
33 1.892946	OppoMobileTe_b9:3a:...	localhost ()	L2CAP	25 Rcvd Information Response (Fixed Channels Supported, Success)

Slika 4: Zmogljivost in indentifikacija naprave

Najprej se izvedejo ukazi **Read Remote Supported Features** in **Read Remote Extended Features**, s katerimi ena naprava pridobi informacije o zmožnostih druge naprave, kot so podprti protokoli, načini prenosa in dodatne funkcionalnosti. Odgovori na te zahteve omogočajo optimizacijo nadaljnje komunikacije.

Sledi ukaz **Remote Name Request**, s katerim naprava zahteva ime ciljne naprave. Ta korak je pomemben za identifikacijo naprave v omrežju.

Po tem se izvedejo še dodatne prilagoditve povezave, kot so:

- **Max Slots Change** – določa število časovnih rež za prenos podatkov,

- **Connection Packet Type Changed** – določa vrsto paketov, ki se bodo uporabljali,
- **Link Supervision Timeout Changed** – nastavi časovno omejitev za zaznavo prekinitve povezave.

Ta faza zagotovi, da sta napravi usklajeni glede zmogljivosti in parametrov povezave, kar omogoča zanesljivo nadaljnjo komunikacijo.

3.1.3 Vzpostavitev L2CAP komunikacije in prenos podatkov

Po zaključeni izmenjavi osnovnih zmogljivosti se komunikacija premakne na višji nivo, kjer se uporablja protokol L2CAP (Logical Link Control and Adaptation Protocol).

24 0.950945	OppoMobileTe_b9:3a:..	localhost ()	L2CAP	15 Rcvd Information Request (Extended Features Mask)
25 0.950953	OppoMobileTe_b9:3a:..	localhost ()	L2CAP	21 Rcvd Information Response (Extended Features Mask, Success)
26 0.950975	localhost ()	OppoMobileTe_b9:3a:..	L2CAP	21 Sent Information Response (Extended Features Mask, Success)
27 0.950988	localhost ()	OppoMobileTe_b9:3a:..	L2CAP	15 Sent Information Request (Fixed Channels Supported)
28 0.956475	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
29 0.986995	OppoMobileTe_b9:3a:..	localhost ()	L2CAP	15 Rcvd Information Request (Fixed Channels Supported)
30 0.986948	localhost ()	OppoMobileTe_b9:3a:..	L2CAP	25 Sent Information Response (Fixed Channels Supported, Success)
31 0.996443	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
32 0.997729	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
33 1.062946	OppoMobileTe_b9:3a:..	localhost ()	L2CAP	25 Rcvd Information Response (Fixed Channels Supported, Success)
34 1.093892	localhost ()	OppoMobileTe_b9:3a:..	L2CAP	57 Sent Echo Request
35 1.074989	OppoMobileTe_b9:3a:..	localhost ()	L2CAP	57 Rcvd Echo Response
36 1.182718	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
37 1.898536	host	controller	HCI_CMD	6 Sent Read RSSI
38 1.898845	controller	host	HCI_EVT	10 Rcvd Command Complete (Read RSSI)
39 1.899016	host	controller	HCI_CMD	7 Sent Read Tx Power Level
40 1.899457	controller	host	HCI_EVT	10 Rcvd Command Complete (Read Tx Power Level)
41 2.075232	localhost ()	OppoMobileTe_b9:3a:..	L2CAP	57 Sent Echo Request
42 2.165237	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
43 2.139946	OppoMobileTe_b9:3a:..	localhost ()	L2CAP	57 Rcvd Echo Response
44 2.899246	host	controller	HCI_CMD	6 Sent Read RSSI
45 2.899614	controller	host	HCI_EVT	10 Rcvd Command Complete (Read RSSI)
46 2.899799	host	controller	HCI_CMD	7 Sent Read Tx Power Level
47 2.900239	controller	host	HCI_EVT	10 Rcvd Command Complete (Read Tx Power Level)
48 3.131371	localhost ()	OppoMobileTe_b9:3a:..	L2CAP	57 Sent Echo Request
49 3.141468	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
50 3.154944	OppoMobileTe_b9:3a:..	localhost ()	L2CAP	57 Rcvd Echo Response
51 3.900108	host	controller	HCI_CMD	6 Sent Read RSSI
52 3.900362	controller	host	HCI_EVT	10 Rcvd Command Complete (Read RSSI)
53 3.900449	host	controller	HCI_CMD	7 Sent Read Tx Power Level
54 3.900729	controller	host	HCI_EVT	10 Rcvd Command Complete (Read Tx Power Level)
55 4.155478	localhost ()	OppoMobileTe_b9:3a:..	L2CAP	57 Sent Echo Request
56 4.188968	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
57 4.194925	OppoMobileTe_b9:3a:..	localhost ()	L2CAP	57 Rcvd Echo Response
58 4.899631	host	controller	HCI_CMD	6 Sent Read RSSI
59 4.900102	controller	host	HCI_EVT	10 Rcvd Command Complete (Read RSSI)
60 4.900201	host	controller	HCI_CMD	7 Sent Read Tx Power Level
61 4.900476	controller	host	HCI_EVT	10 Rcvd Command Complete (Read Tx Power Level)
62 5.195178	localhost ()	OppoMobileTe_b9:3a:..	L2CAP	57 Sent Echo Request
63 5.218079	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
64 5.227449	OppoMobileTe_b9:3a:..	localhost ()	L2CAP	57 Rcvd Echo Response

Slika 5: L2CAP komunikacija

V začetni fazi si napravi izmenjata **Information Request** in **Information Response** pakete. Ti paketi vsebujejo informacije o podprtih L2CAP funkcionalnostih, kot so razširjene zmožnosti (Extended Features Mask) in podprti fiksni kanali (Fixed Channels Supported). Na podlagi teh informacij napravi določita način logične komunikacije ter parametre prenosa podatkov.

Ko je L2CAP plast ustrezno konfigurirana, se začne dejanska komunikacija med napravama. V našem primeru gre za pošiljanje *ping* paketov z uporabo L2CAP Echo Request in Echo Response mehanizma.

Naprava pošlje paket **Echo Request**, ki vsebuje določeno podatkovno vsebino (payload). Ciljna naprava na ta paket odgovori z **Echo Response**, pri čemer vrne enako vsebino nazaj pošiljatelju. Ta mehanizem omogoča preverjanje dosegljivosti naprave in merjenje zakasnitve v komunikaciji.

Na podlagi časovne razlike med poslanim zahtevkom in prejetim odgovorom lahko izračunamo čas krožnega potovanja (RTT – Round Trip Time), ki predstavlja pomemben kazalnik zmogljivosti povezave.

V zajetem prometu opazimo več zaporednih parov **Echo Request** in **Echo Response**, kar ustreza izvajanju ukaza **l2ping**. Poleg tega se med posameznimi zahtevki izvajajo tudi dodatni ukazi, kot sta **Read RSSI** in **Read TX Power Level**, ki omogočajo spremljanje kakovosti povezave med komunikacijo.

Ta faza predstavlja glavni del prenosa podatkov in praktično uporabo vzpostavljene Bluetooth povezave.

Na koncu velja omeniti, da se lahko pri analizi L2CAP komunikacije v določenih primerih pojavi tudi paket **Echo Response [Malformed Packet]**. Tak zapis pomeni, da Wireshark zazna nepravilno strukturiran ali nepopolno dekodiran paket, kar je lahko posledica napake v implementaciji Bluetooth sklada na napravi, nepopolnega zajema prometa ali nezdružljivosti med napravami.

V analizirani komunikaciji z napravo OPPO Find X5 takšnih paketov nismo zaznali, kar kaže na pravilno in standardno implementacijo protokola. Vendar pa sem med testiranjem na napravi Samsung Galaxy Ace S5830 opazil pojav **Malformed Packet** pri **Echo Response**, kar nakazuje na morebitne omejitve ali nepravilnosti v starejši Bluetooth implementaciji te naprave.

46 3.546511	localhost ()	SamsungElect_62:d3:...	L2CAP	57 Sent Echo Request
47 3.548911	controller	host	HCI_EVT	8 Rcvd Number of Completed Packets
48 3.582060	SamsungElect_62:d3:...	localhost ()	L2CAP	13 Rcvd Echo Response[Malformed Packet]
49 4.443855	host	controller	HCI_CMD	6 Sent Read RSSI
50 4.444926	controller	host	HCI_EVT	10 Rcvd Command Complete (Read RSSI)

Slika 6: Napaka v komunikaciji z Samsung Galaxy Ace S5830

3.1.4 Zaključek povezave

Po končani komunikaciji se povezava med napravama ustrezno prekine. Ta proces se začne z ukazom **Disconnect**, ki ga gostitelj (host) pošlje krmilniku z namenom prekinitve obstoječe povezave.

111 12.905827	controller	host	HCI_EVT	10 Rcvd Command Complete (Read Tx Power Level)
112 13.667330	host	controller	HCI_CMD	7 Sent Disconnect
113 13.667571	controller	host	HCI_EVT	7 Rcvd Command Status (Disconnect)
114 13.777995	controller	host	HCI_EVT	7 Rcvd Disconnect Complete

Slika 7: Prekinitev povezave

Najprej se pojavi paket **Command Status (Disconnect)**, ki potrdi, da je krmilnik sprejel zahtevo za prekinitev povezave. Nato sledi paket **Disconnect Complete**, ki označuje, da je bila povezava uspešno prekinjena.

Ta faza zagotavlja pravilno in kontrolirano zapiranje povezave, s čimer se sprostijo uporabljeni viri in preprečijo morebitne napake ali nedoslednosti v nadaljnji komunikaciji.

4 Tipi napadov na bluetooth naprave

4.1 BlueSnarfing

BlueSnarfing je neavtoriziran dostop do informacij na brezžični napravi prek Bluetooth povezave [10]. Ta napad omogoča pridobitev dostopa do koledarja, telefonskega imenika, e-pošte, SMS sporočil in drugih osebnih podatkov. Pogosto izkorišča profil OBEX Push (Object Exchange), ki ob nepravilni konfiguraciji razkrije podatke brez potrditve uporabnika.

4.2 MAC address spoofing

Ta napad se lahko zgodi, ko se napravi dogovarjata za varne ključe, s katerimi bosta komunicirali, torej mora biti napadalec že na začetku prisoten. Napadalec ponaredi (spoofa) MAC naslov zaupanja vredne naprave in tako preliči ciljno napravo, da se poveže z njegovo namesto s prvotno napravo. S tem lahko prestreže komunikacijo ali se lažno avtenticira, ne da bi razkril svoj pravi naslov.

4.3 BlueBorne

BlueBorne je kolekcija več Bluetooth napadov, ki izkoriščajo napake v Bluetooth kopici (stack), zlasti v BlueZ (Linux), Androidu, Windows in iOS. Z uporabo prekoračitve medpomnilnika (buffer overflow) ali drugih pomnilniških napak lahko napadalec na ciljni napravi izvede poljubno nezaželeno kodo, brez da bi bil seznanjen z napravo ali da bi ta zahtevala paring.

4.4 Reflection Attack

Do reflection napada pride, ko napadalec prestreže avtentikacijske podatke (npr. kriptografske izzive in odgovore) med dvema napravama in jih nato ponovi (reflektira) tretji napravi ali isti napravi v drugem času. S tem ne razkrije nobenih lastnih informacij in lahko pridobi neavtoriziran dostop do storitev, kot da bi bil legitimen uporabnik, medtem ko se predstavlja za nekoga drugega.

4.5 Denial of Service (DoS)

Pri tem napadu napadalec poskuša prekiniti povezavo med dvema napravama ali narediti Bluetooth vmesnik popolnoma neuporaben. To doseže s pošiljanjem velikega števila paketov (npr. zahtevkov L2CAP, SDP ali praznih paketov), kar povzroči preobremenitev in sesutje Bluetooth kopice. Posledično naprava ne more več vzpostavljati novih povezav ali pa se obstoječe prekinejo.

4.6 Bluejacking

Bluejacking je pošiljanje nezaželenih sporočil prek Bluetootha. Napadalec uporabi protokol OBEX (lahk komunikacijski protokol za pošiljanje binarnih datotek), da pošlje kratko sporočilo, ki se prikaže na zaslonu žrtve. Čeprav je bluejacking večinoma nedolžna potegavščina, se lahko zlorabi za namene socialnega inženiringa ali širjenja zlonamerne vsebine.

4.7 Bluesmacking

Bluesmacking je oblika DoS napada, specifična za Bluetooth. Napadalec pošlje veliko število lažnih zahtevkov L2CAP echo (ping) na ciljno napravo, ki jih ta ne more pravilno obdelati. To povzroči preobremenitev procesorja in posledično izpad Bluetooth storitev.

4.8 PIN Brute Force

Pri starejših napravah z Bluetooth 2.0 in nižje se uporablja kratek PIN (običajno 4–6 števk). Napadalec lahko s prisluškovanjem parjenju prestreže izmenjane pakete in nato s programom (npr. 'bterack') poskusi vse možne kombinacije PIN kode. Ko jo odkrije, lahko izračuna skupne ključe in dešifrira celotno komunikacijo.

5 Eksperimentalna analiza Bluetooth napadov

V tem delu smo preučili različne bluetooth napade na različnih napravah.

Najprej pa rabimo spoznati par orodij, ki jih bomo uporabljali poleg teh napadov.

5.1 Hcitol

`hcitol` je ukaz v Linuxu, ki omogoča upravljanje in nadzor Bluetooth povezav na nizki ravni (Host Controller Interface - HCI). Je del paketa `BlueZ`, najbolj razširjene Bluetooth implementacije za Linux, ki mu omogoča neposredno komunikacijo s strojno opremo (adapterjem). V spodnji tabeli so predstavljeni najpogostejše uporabljeni ukazi, ki jih bom uporabljal v napadih in njihov namen.

Ukaz	Opis
<code>hcitol dev</code>	Prikaže seznam lokalnih Bluetooth adapterjev (npr. <code>hci0</code>).
<code>hcitol scan</code>	Skenira vse odkrivajoče se naprave v dosegu in izpiše njihov MAC naslov ter ime.
<code>hcitol info <MAC></code>	Pridobi podrobne informacije (ime, razred, LMP verzijo, značilnosti).
<code>hcitol rssi <MAC></code>	Prikaže jakost signala (RSSI) za povezano napravo.
<code>hcitol lq <MAC></code>	Prikaže kakovost povezave (Link Quality).
<code>hcitol lescan</code>	Skenira BLE (Bluetooth Low Energy) naprave.
<code>hcitol leinfo <MAC></code>	Pridobi informacije o BLE napravi.

Tabela 3: Pregled ukazov orodja `hcitol`

5.2 Bluetoothctl

`bluetoothctl` je sodobno interaktivno orodje za upravljanje Bluetooth naprav v Linuxu. Prav tako je del paketa `BlueZ`, vendar za razliko od zastarelega `hcitol` ponuja višjenivojski vmesnik, ki podpira vse sodobne Bluetooth profile in storitve, vključno s poenostavljenim seznanjanjem (pairing), zaupanjem naprav (trust) in upravljanjem BLE naprav. Uporablja se v interaktivnem načinu, kjer uporabnik vnaša ukaze v lastno lupino.

V spodnji tabeli so predstavljeni najpogostejše uporabljeni ukazi znotraj

`bluetoothctl`.

Ukaz	Opis
<code>power on</code>	Vklopi Bluetooth adapter.
<code>power off</code>	Izklopi Bluetooth adapter.
<code>scan on</code>	Začne skenirati naprave v dosegu.
<code>scan off</code>	Ustavi skeniranje.
<code>devices</code>	Prikaže seznam vseh naprav, ki jih je adapter zaznal.
<code>pair <MAC></code>	Začne postopek seznanjanja z napravo z danim MAC naslovom.
<code>trust <MAC></code>	Označi napravo kot zaupanja vredno (samodejno dovoli povezave brez ponovnega seznanjanja).
<code>connect <MAC></code>	Vzpostavi povezavo s predhodno seznanjeno napravo.
<code>disconnect <MAC></code>	Prekine povezavo z napravo.
<code>remove <MAC></code>	Odstrani napravo iz seznama seznanjenih (pozabi jo).
<code>info <MAC></code>	Prikaže podrobne informacije o napravi (ime, razred, UUID, stanje povezave).
<code>show</code>	Prikaže informacije o lokalnem Bluetooth adapterju.
<code>agent on</code>	Vklopi agenta za upravljanje PIN kod in potrditev parjenja.
<code>default-agent</code>	Nastavi trenutnega agenta kot privzetega.
<code>exit</code>	Zapre <code>bluetoothctl</code> .

Tabela 4: Pregled ukazov orodja `bluetoothctl`

Orodje `bluetoothctl` je danes standardni način za upravljanje Bluetooth povezav v Linuxu in popolnoma nadomešča starejša orodja, kot so `hcitool`, `rfcomm` in `sdptool`. Omogoča tudi delo z BLE napravami prek podukazov, kot sta `ble` in `advertise`.

5.3 Sdptool

`sdptool` je ukaz v Linuxu, ki omogoča poizvedovanje in upravljanje strežnikov protokola SDP (Service Discovery Protocol). Je del paketa `BlueZ` in omogoča iskanje specifičnih storitev na oddaljenih napravah ter pregled vseh razpoložljivih storitev prek ukaza `browse`. SDP je ključnega pomena za odkrivanje storitev, ki jih naprava ponuja (npr. prenos datotek, dostop do telefonskega imenika, brezžični zvok). V spodnji tabeli je predstavljen najpomembnejši ukaz orodja `sdptool`.

Orodje `sdptool` je danes označeno kot zastarelo in ga v sodobnih distribucijah Linuxa nadomeščata orodji `bluetoothctl` in `btmgmt`. Kljub temu pa

Ukaz	Opis
<code>sdptool browse <MAC></code>	Pregled vseh razpoložljivih storitev na oddaljeni napravi z danim MAC naslovom. Izpiše imena storitev, kanale in profile.

Tabela 5: Pregled ukazov orodja `sdptool`

ostaja uporabno za poglobljeno diagnostiko in razumevanje protokola SDP, zlasti pri starejših napravah in sistemih.

5.4 Še nekaj osnovnih ukazov

Ukaz	Opis
<code>hciconfig -a</code>	Prikaže podrobne informacije o vseh Bluetooth vmesnikih (npr. <code>hci0</code>), vključno z MAC naslovom, stanjem (UP/DOWN), različico HCI, imenom naprave in razredom.
<code>hciconfig hci0 up</code>	Aktivira (vklopi) Bluetooth vmesnik <code>hci0</code> . Po tem ukazu je adapter pripravljen za skeniranje in povezovanje.
<code>hciconfig hci0 down</code>	Deaktivira (izklopi) Bluetooth vmesnik <code>hci0</code> .
<code>hciconfig hci0 reset</code>	Ponastavi Bluetooth vmesnik <code>hci0</code> . Uporabno, kadar se adapter ne odziva pravilno.
<code>hciconfig hci0 name "ime"</code>	Spremeni lokalno ime Bluetooth naprave (prikazano ob skeniranju).
<code>systemctl start bluetooth</code>	Zažene Bluetooth storitev (oz. <code>bluetooth.service</code>) v sistemu.
<code>systemctl stop bluetooth</code>	Ustavi Bluetooth storitev.
<code>systemctl restart bluetooth</code>	Ponovno zažene Bluetooth storitev – uporabno po spreminjanju konfiguracije ali odpravljanju težav.
<code>systemctl enable bluetooth</code>	Nastavi, da se Bluetooth storitev samodejno zažene ob zagonu sistema.
<code>systemctl status bluetooth</code>	Prikaže trenutno stanje Bluetooth storitve (ali je aktivna, morebitne napake).

Tabela 6: Pregled ukazov `hciconfig` in `systemctl` za Bluetooth

Sedaj smo pregledali vse osnovne ukaze in kako se dela z bluetoothom na linux napravah, tako da lahko začnemo z dejanskimi napadi.

5.5 CVE-2023-45866

Prvi napad, ki sem ga izvedel, temelji na ranljivosti CVE-2023-45866. V sklopu BlueZ lahko neavtenticirana HID (Human interface device) naprava v vlogi periferne enote vzpostavi šifrirano povezavo in pošilja poročila HID tipkovnice, ne da bi centralna naprava zahtevala kakršno koli interakcijo uporabnika. S tem je možno injicirati HID sporočila (npr. pritiske tipk) brez vednosti uporabnika. Ta napad sem testiral na treh telefonskih napravah in sicer:

- OPPO Find X5 (bluetooth 5.3)
- Samsung Galaxy S7 edge (bluetooth 4.2)
- Samsung Galaxy Ace S5830 (bluetooth 3.0)

Mogoče poznate Rubber Ducky, to je USB ključek, ki na površini zgleda kot vsak drug ključek, ampak ko ga priključiš v računalnik se predstavlja kot da je human interface device (npr. tipkovnica) in začne izvajati neko kodo, ker je računalnik narejen tako da zaupa človeku to kodo izvede. Kaj če bi to lahko naredil kdorkoli samo z dostopom do bluetootha? Izkaže se da s CVE-2023-45866 lahko točno to dosežemo. Napisal sem 2 bolj enostavna skripta za Rubber ducky in uporabil malo prilagojeno kodo BlueDucky [7, 4] en od teh skriptov je bolj satiričen in sicer predvaja pesem Rick Astley - Never gonna give you up, drugi pa gre v nastavitve telefona.

```
REM Rickroll Android  
DELAY 2000
```

```
REM Open Chrome  
GUI b  
DELAY 500  
CTRL l  
STRING https://www.youtube.com/watch?v=dQw4w9WgXcQ  
ENTER  
DELAY 3000
```

```
REM Go to settings  
DELAY 2000  
GUI i  
DELAY 500
```

Seveda ukaz kot je GUI i je na vsakem telefonu bil drugačen. Ta specifičen skript je napisan za OPPO Find X5, pri katerem je sistemski jezik v italijanščini.

5.5.1 Rezultati testiranja

Za vsako od treh naprav sem poskusil izvesti dva različna DuckyScripta: prvi je prek brskalnika Chrome predvajal Rickroll video, drugi pa je poskušal odpreti sistemske nastavitve. Rezultati so prikazani v tabeli 7.

Naprava	BT razli-čica	Rickroll	Odpiranje nastavitev
OPPO Find X5	5.3	Uspešno	Uspešno
Samsung Galaxy S7 edge	4.2	Uspešno	Uspešno
Samsung Galaxy Ace S5830	3.0	Neuspešno	Neuspešno

Tabela 7: Uspešnost injiciranja tipk prek CVE-2023-45866 na testiranih napravah

Pri obeh novejših napravah (OPPO Find X5 in Samsung S7 edge) je napad deloval brez vsakršnega opozorila ali zahteve po potrditvi, vendar le zato, ker sta bili napravi že predhodno seznanjeni (paired) z mojim Bluetooth adapterjem. Telefon je takoj sprejel povezavo s HID tipkovnico in izvedel vnešene ukaze, saj je zaupanje med napravama že obstajalo.

Omeniti velja, da je bila ranljivost CVE-2023-45866 v času testiranja že uradno popravljena (patched). Pred popravkom pa napad sploh ni zahteval predhodnega seznanjanja – dovolj je bilo, da je napadalec vzpostavil HID povezavo brez kakršne koli predhodne interakcije z uporabnikom.

Kaj pa če bi želeli celo da naprava ne bi nikoli (niti enkrat v preteklosti) potrdila zahtevka za parjanje? Ali bi bilo mogoče s katerokoli napravo doseči, da telefon oziroma druga naprava dovoli dostop tudi po tem, ko je bila ranljivost že odpravljena (patchana)? Žalosten odgovor je, da je to še kako možno. Za to rabimo še en napad, ki sem ga opisal na začetku in sicer MAC address spoofing. S tem smo ponaredili MAC address naprave, ki smo jo enkrat že potrdili (npr. brezžične slušalke ali pametna ura), ker je bila ta naprava že potrjena se telefon brez kakršne koli nove potrditve poveže in izvede kodo. Moremo poudariti, da je MAC spoofing zelo preprost, sicer ne podpira vsak hardware ampak podpira pa skori vsak USB ključek, ki ga

kupiš v kakršnikoli trgovini, ki prodaja elektroniko. Ta spoofing napad sem tudi lahko naredil z enim samim ukazom:

```
spooftooph -i hci1 -a 2C:4D:79:78:2F:D7 -n "OPPO Enco X2"
```

Če pa ima napadalec naprednejšo opremo, kot je Ubertooth One, lahko brez večjih težav pridobi MAC naslov naprav v okolici. Na primer, ko so slušalke že povezane s telefonom, običajno ne oglašujejo več svoje prisotnosti, zato jih druge naprave ne zaznajo. Z napravo Ubertooth One pa je mogoče prisluškovati Bluetooth prometu in iz izmenjave podatkov razbrati MAC naslov, ki ga je nato mogoče uporabiti pri nadaljnjih napadih.

Pri najstarejši napravi, Samsung Galaxy Ace S5830 (Bluetooth 3.0), napad ni uspel. Telefon se ni odzval na poskuse vzpostavitve HID povezave. Razlog je v tem, da ta naprava podpira le zelo omejen nabor Bluetooth storitev in profilov – med drugim ne ponuja profila HID (Human Interface Device) prek Bluetootha. Poleg tega ima odprtih le nekaj RFCOMM kanalov (predvsem za Headset in Handsfree), zato preprosto ni imela ustreznega vstopnega mesta za tovrstni napad.

5.5.2 Ugotovitve

Na podlagi opravljenih testov lahko zaključim naslednje:

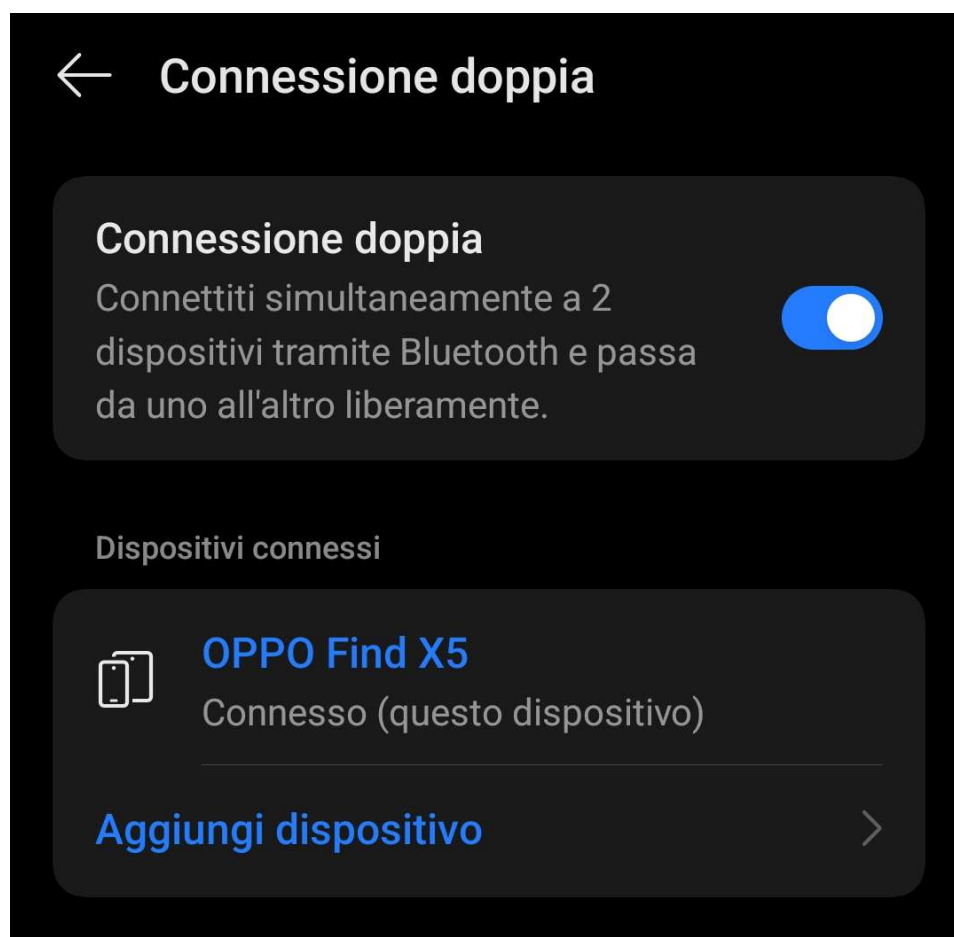
- Ranljivost CVE-2023-45866 omogoča injiciranje tipk na vseh novejših Android napravah (Bluetooth 4.2 in novejši), ki niso bile popravljene in z malo več dela tudi na napravah, ki so načeloma bile popravljene. Napad poteka popolnoma brez interakcije uporabnika – žrtev ne vidi nobenega obvestila o parjenju ali povezovanju.
- Čeprav je bilo za prvo povezavo včasih treba potrditi dovoljenje za seznanitev (pairing), smo tudi to oviro lahko zaobšli s pomočjo MAC spoofing-a.
- Starejše naprave (Bluetooth 3.0 in nižje) praviloma niso ranljive za ta napad, ker ne podpirajo HID profila prek Bluetootha oziroma nimajo dovolj odprtih storitev, prek katerih bi lahko vzpostavili takšno povezavo. Seveda to ne pomeni da so ti protokoli kakorkoli varni pred drugimi napadi.

Napad CVE-2023-45866 tako predstavlja resno varnostno tveganje za vse Android naprave, saj napadalcu omogoča popoln nadzor nad vnosom tipk – od odpiranja zlonamernih spletnih strani do izvajanja sistemskih ukazov.

5.6 Denial of Service (DoS)

V tem napadu bom poskušal prekiniti povezavo med dvema napravama, in sicer med brezžičnimi slušalkami OPPO Enco X2 in telefonom OPPO Find X5. To bom poskušal doseči z ukazom `l2ping`, kasneje pa bom ta ukaz izvajal vzporedno (z več nitmi), da bom poslal večje število paketov.

Najprej se pogovorimo o pripravi na ta napad. Veliko boljših brezžičnih slušalk ima zaščito pred tovrstnimi napadi, med njimi tudi OPPO Enco X2. Te se varujejo tako, da preprosto zavržejo vse pakete, ki ne prihajajo iz naprave, s katero so trenutno povezane. Seveda bi se z dovolj veliko količino paketov tudi ta zaščita lahko sesula. Ena dobrih lastnosti teh slušalk pa je funkcija, ki dovoljuje, da sta nanje hkrati povezani dve napravi. To je lahko uporabno v zelo specifičnih primerih – na primer, če si s partnerjem izmenjujeta predvajanje glasbe – toda nam odpre možnost za naš DoS napad.



Slika 8: Nastavitve slušalk, ki dovoljujejo dvojno povezavo

Tako sem se najprej s telefonom povezal na slušalke, nato pa poskušal izvesti DoS napad z ukazom `l2ping`.

Najprej sem preizkusil ukaz:

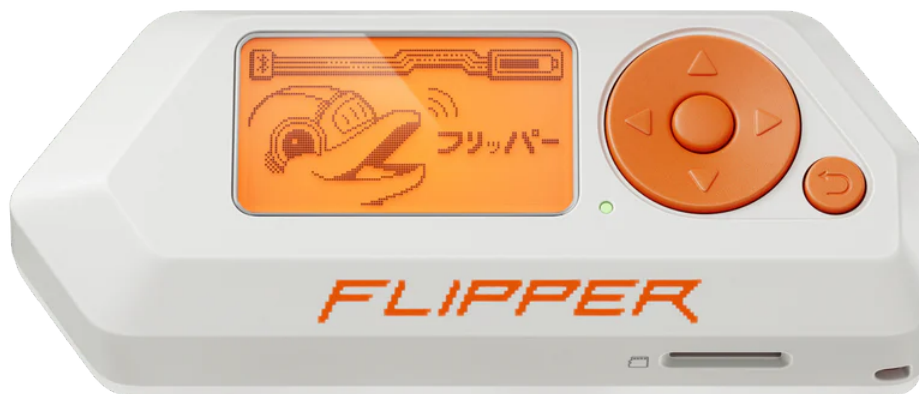
```
l2ping -i hci0 -s 600 -f 2C:4D:79:78:2F:D7
```

Ta konstantno pošilja ping pakete z velikostjo 600 bajtov in čim manjšo zakasnitvijo (to omogoča zastavica `-f`). Na ta napad so bile slušalke popolnoma odporne, saj imajo očitno dovolj procesorske moči, da odgovorijo na vsak tak L2CAP echo paket. Zato je bilo treba napad izboljšati – to smo dosegli z multithreadingom, tj. z zagonom več takih ukazov hkrati. Pri tem smo si pomagali s python skripto `badblue.py`. Ta zažene 300 niti (threadov) z enakim ukazom kot prej in tako uspešno sesuje povezavo med telefonom in slušalkami.

Napad sem preizkusil tudi na telefonu OPPO Find X5 in Samsung Galaxy Ace S5830. Na OPPO Find X5 ni imel vpliva, medtem ko je imel Samsung Galaxy Ace S5830 velike težave z Bluetooth povezavo. Na moje ICMP pakete je odgovarjal z zamikom do 15000 ms oziroma 15 s, kar je povezavo naredilo praktično neuporabno.

5.7 Bluetooth spam

Ta napad je zelo podoben prejšnjemu napadu in bi ga tudi štel pod denial of service, ampak bi mu rad dal dodatno pozornost. V tem delu bom uporabljal posebno opremo in sicer **Flipper zero** uporabil pa ga bom da zamrznem iPhone napravo (iPhone 8).



Slika 9: Flipper zero

Napad Bluetooth spam s Flipper Zero izkorišča ranljivost v načinu za združevanje (pairing) naprav. Flipper Zero se lažno predstavi kot širokopasovna naprava HID (npr. tipkovnica, miška ali slušalke) in nato neprekinjeno pošilja zahteve za vzpostavitev povezave prek protokola L2CAP na ciljno napravo. Vsaka zahteva sproži na tarči celoten postopek odkrivanja storitev in avtentikacije, ki porablja sistemske vire. Ker Flipper Zero pošilja več sto zahtevkov na sekundo, postane Bluetooth sklad preobremenjen – pojavijo se zakasnitve, zamrznitev uporabniškega vmesnika in na koncu popolna blokada naprave.

Napad ne prizadene samo iPhonov, temveč tudi druge naprave z operacijskimi sistemi Android in starejše različice macOS. Še posebej ranljive so bile različice iOS od 12.0 do 17.1, kjer je napad povzročil, da je telefon postal **popolnoma neuporaben** – ni se odzival na dotike, gumbi niso delovali, edina rešitev pa je bil trd ponovni zagon (force restart). Kasneje je Apple v posodobitvi iOS 17.2 uvedel zaščito proti tej specifični ranljivosti, vendar določeni modeli (zlasti starejši iPhone 6, 7, 8 in X) ostajajo delno ranljivi,

saj ne morejo namestiti najnovejšega iOS-a. Prav tako podobne težave še vedno obstajajo na nekaterih napravah Android z Bluetooth sklado, ki nima vgrajenih zaščit.

6 Zaščita pred Bluetooth napadi

Na podlagi izvedenih eksperimentov in opisanih napadov lahko podamo smernice za zaščito pred Bluetooth ranljivostmi. Zaščita je odvisna tako od uporabnikovih navad kot od strojne in programske opreme naprave.

6.1 Splošni ukrepi

- **Izklop Bluetootha, ko ni v uporabi** – najučinkovitejša zaščita pred vsemi napadov. Napadalec se ne more povezati z napravo, če je Bluetooth onemogočen.
- **Nevidni način (non-discoverable)** – naprava se ne oglašuje, kar oteži začetno zaznavo, sicer s pravo opremo (npr. **Ubertooth One**) to velikokrat ni zadosti. Večina napadov zahteva, da je tarča odkritljiva vsaj v času parjenja.
- **Redna posodabljanja programske opreme** – proizvajalci pogosto zakrpajo znane ranljivosti (npr. CVE-2023-45866, BlueBorne). Redno nameščanje najnovejših varnostnih popravkov za operacijski sistem in Bluetooth sklad zelo zmanjša možnosti za možne napade ali pa napad vsaj oteži.
- **Uporaba sodobnih Bluetooth različic** – naprave z Bluetooth 4.2+ in vključenimi *Secure Connections* so odporne na napade PIN brute force in reflection, saj uporabljajo močnejšo kriptografijo (ECDH) in zaščito pred ponovitvami.

6.2 Specifične zaščite glede na vrsto napada

Napad	Zaščita
BlueSnarfing	Onemogočite profil OBEX Push, če ga ne potrebujete. Zahtevajte avtentikacijo za vsak dostop do osebnih podatkov.
MAC address spoofing	Uporabljajte Secure Simple Pairing (SSP) z numerično primerjavo ali out-of-band (npr. NFC). Izogibajte se legacy pairing (PIN koda).
BlueBorne	Posodobite BlueZ (Linux), Android (popravek od avgusta 2017), Windows (popravek od septembra 2017) in iOS (iOS 10+).
Reflection Attack	Uporabite medsebojno avtentikacijo s časovnimi žigi ali naključnimi števili (nonce). Sodobni protokoli (BLE Secure Connections) to že vključujejo.
DoS (l2ping, Bluesmacking)	Na napravi omogočite omejevanje hitrosti L2CAP zahtevkov. Posodobite vdelano programsko opremo (firmware) Bluetooth čipa.
Bluejacking	Ne odpirajte sumljivih sporočil prek OBEX. Onemogočite sprejemanje datotek brez potrditve.
PIN Brute Force	Nikoli ne uporabljajte kratkih PIN kod (0000, 1234). Če naprava podpira SSP, izklopite legacy pairing.
CVE-2023-45866 (HID injection)	Namestite varnostni popravek (BlueZ 5.68+ ali Android varnostni popravek za december 2023). Po popravku naj bi napad zahteva predhodno seznanitev – zato ne sprejemajte parjenja z neznanimi napravami .
Bluetooth spam (Flipper Zero)	Posodobite iOS na različico 14.8+ ali 15+ (Apple je uvedel omejitev hitrosti parjenja). Na Androidu vklopite <i>Bluetooth snooping</i> filter (če je na voljo).

Tabela 8: Pregled zaščitnih ukrepov glede na vrsto napada

6.3 Posebnosti za starejše naprave (Bluetooth 3.0 in nižje)

Starejše naprave (npr. Samsung Galaxy Ace S5830) so ranljive za PIN brute force, reflection in DoS, saj ne podpirajo sodobnih varnostnih mehanizmov. Za takšne naprave je edina zanesljiva zaščita **popoln izklop Bluetootha**,

razen kadar je povezava nujno potrebna. Prav tako jih ne uporabljajte za občutljive podatke, če je Bluetooth vklopljen. Seveda take naprave imajo tudi dosti manj odprtih storitev preko bluetootha, tako da niso vsi napadi možni, ampak preko take povezave ne nikoli prenašati kakšnih občutljivih podatkov.

7 Zaključek

V seminarski nalogi smo celovito raziskali področje varnosti Bluetooth tehnologije, od osnovnih meritev signala do naprednih metod vdora. Skozi praktično delo smo ugotovili, da moč signala (RSSI) sicer upada z razdaljo, vendar število naprav v okolici ne vpliva nujno na moč, temveč predvsem na variabilnost in stabilnost povezave.

Analiza prometa z orodjem Wireshark je omogočila podroben vpogled v L2CAP protokol in handshake proces. Ugotovljeno je bilo, da lahko že majhne nepravilnosti v implementaciji protokola (kot so bili *malformed packets* pri starejšem Samsungu) povzročijo težave v komunikaciji, kar lahko napadalcu izkoristijo za DoS napade.

Najučinkovitejša in hkrati najbolj skrb vzbujajoča je bila demonstracija napada CVE-2023-45866. Dejstvo, da lahko napadalec brez kakršne koli interakcije uporabnika prevzame nadzor nad vnosom tipkovnice, kaže na resnost Bluetooth ranljivosti. Čeprav so sodobni sistemi načeloma popravljeni, kombinacija s tehnikami, kot je MAC spoofing, dokazuje, da popolne varnosti ni – še posebej, če napadalec izkoristi vnaprej vzpostavljeno zaupanje med napravami.

Naloga nam je pokazala, da je bluetooth izjemno priročen, a hkrati varnostno tvegan protokol. Za povprečnega uporabnika je ključno zavedanje, da Bluetooth vmesnik predstavlja odprta vrata v napravo, ki jih je treba odpirati pazljivo in le takrat, ko je to nujno potrebno. Pridobljeno znanje bo služilo kot osnova za nadaljnje raziskovanje brezžičnih tehnologij in kibernetske varnosti.

Literatura

- [1] DevAcademy. Bluetooth security fundamentals. <https://academy.nordicsemi.com/courses/bluetooth-low-energy-fundamentals/lessons/lesson-5-bluetooth-le-security-fundamentals/topic/pairing-process/>. Razlaga pairing procesa.
- [2] Wireshark Foundation. Wireshark. <https://www.wireshark.org/>. Uradna spletna stran priljubljenega orodja za analizo omrežnega prometa.
- [3] Jason Marcel. Bluetooth adaptive frequency hopping. <https://www.bluetooth.com/blog/how-bluetooth-technology-uses-adaptive-frequency-hopping-to-overcome-packet-interference/>. Članek o bluetooth AFH.
- [4] National Vulnerability Database (NVD). Cve-2023-45866. <https://nvd.nist.gov/vuln/detail/CVE-2023-45866>. Ranljivost v BlueZ (Linux Bluetooth sklad), ki omogoča neavtenticirani HID napravi vzpostavitev šifrirane povezave in injiciranje pritiskov tipk brez posredovanja uporabnika.
- [5] National Vulnerability Database (NVD). Cve-2025-20695. <https://nvd.nist.gov/vuln/detail/CVE-2025-20695>. Ranljivost v Bluetooth vdelani programski opremi (firmware) – oddaljena zavrnitev storitve (DoS) zaradi neujete izjeme. Prizadete naprave z Android 13–15 in Mediatek SDK.
- [6] Husain Parvez and Ata Hakçıl. What is bluebugging? how bluetooth attacks work and how to prevent them. <https://www.expressvpn.com/blog/bluebugging/>. Podroben opis bluebugging napada, njegovega delovanja, zgodovine ter zaščite pred njim.
- [7] pentestfunctions. Blueducky. <https://github.com/pentestfunctions/BlueDucky>. Orodje za izkoriščanje ranljivosti CVE-2023-45866 za injiciranje pritiskov tipk prek Bluetooth HID.
- [8] Red Hat Customer Portal. Bleedingtooth – kernel bluetooth vulnerabilities. <https://access.redhat.com/security/vulnerabilities/BleedingTooth>. Zbirka napak v Linux kernelu (CVE-2020-12351, CVE-2020-12352, CVE-2020-24490), ki omogočajo padec sistema, izvedbo poljubne kode ali uhajanje podatkov.
- [9] Bluetooth Special Interest Group (SIG). Bluetooth l2cap connection establishment and handshake process. <https://www.bluetooth.com/specifications/specs/core-specification-5-3/>. Uradna specifikacija Bluetooth Core, ki podrobno opisuje L2CAP protokol in vzpostavitev povezave.

- [10] Wikipedia. Bluesnarfing. <https://en.wikipedia.org/wiki/Bluesnarfing>.
Nepooblaščen dostop do podatkov prek Bluetooth povezave z izkoriščanjem protokola OBEX.